PACT

Household trust controls for action-capable AI

Full technical documentation | Alexa+ track | AWS Builder mini-challenge | No hardware required

VOICE-FIRST	DETERMINISTIC	VERIFIED
Simulated Alexa+ experience	Code decides, not the LLM	240 adversarial attempts, 0 bypasses



The model proposes. The household confirms. The policy engine decides. The executor verifies. The receipt remembers.

This document explains the product, architecture, security model, verified implementation, demo flow, and honest limitations. The enforcement core is deliberately independent of the visual firefly.

Prepared October 17, 2026

How to understand PACT in one minute

PACT is not an AI that decides whether a request is safe. It is a trust boundary around an AI that can act. Bedrock understands household language and proposes a policy. A person confirms it. Deterministic code evaluates every action. A signed token binds the decision to the exact request, and a receipt records what happened.

1. Delegate

"Ask me before booking anything over $100." Bedrock turns the sentence into a candidate rule and reads it back.

2. Enforce

A mock add-on submits a canonical request. PACT returns allow, ask, or deny. The language model is not in this decision path.

3. Verify

An approval and execution token are bound to the exact amount, merchant, person, surface, policy version, and one-time nonce.

4. Remember

Every attempt creates a tamper-evident receipt, including denied, replayed, stale, and parameter-tampered attempts.

The central demonstration is Maya's birthday plan: VenueBook requests a $120 booking and waits for Dad; GiftCart then requests $40 of decorations and is denied because approved plus pending exposure would be $160 against a $150 plan budget.

Document structure

The remainder is the full project record: product rationale, rule and action contracts, MCP and Bedrock integration, token and receipt security, threat model, test evidence, voice and visual interaction, deployment boundaries, roadmap, and submission claims.

1. Executive summary

PACT is a household rulebook and authorization gateway for AI assistants that can take actions on behalf of people.

A household member can say:

> *"Ask me before booking anything over $100."*

PACT uses Amazon Bedrock to turn that sentence into a candidate rule and reads the interpretation back in plain language. The rule is not active until a person confirms it. From that point onward, every action-capable add-on submits a structured request to PACT before it acts.

The authorization decision is made by deterministic application code-not by the language model. The result is one of three decisions:

- **Allow:** the action can proceed.
- **Ask:** a named household approver must approve the exact request.
- **Deny:** the action cannot proceed.

The executor accepts an action only when it carries a valid, short-lived, HMAC-signed decision token bound to the exact request and current policy version. Every attempt-including denied, pending, approved, tampered, and replayed attempts-creates a tamper-evident demo receipt.

The hackathon demo uses simulated household identity, mock add-ons, and mock actions. It does not intercept real Alexa+ actions, charge real accounts, unlock a real door, or prove production-wide enforcement. The prototype demonstrates a policy gateway that Amazon could standardize for Alexa+ add-ons.

Product promise

> ***Give a household one understandable rulebook for what an AI assistant may do, what it must ask, and what it must never do.***

Amazon-facing thesis

Alexa+ is moving from answering questions toward completing tasks. As the assistant gains the ability to purchase, book, control, unlock, and reveal information, the household needs a single understandable consent and enforcement layer. PACT is a prototype of that layer.

The short pitch

> *"PACT gives Alexa+ add-ons a common household permission contract. Bedrock helps translate household language into a rule; code enforces it; every action leaves a receipt."*

---

2. The problem

Conversational agents are becoming capable of acting rather than merely responding. That changes the trust problem.

A user does not only need to know whether an agent can perform an action. They need to know:

1. **Who is asking?**
2. **What is the assistant being asked to do?**
3. **What is this person allowed to do?**
4. **When should an adult or account owner approve it?**
5. **What information may appear on a shared surface?**
6. **What happened after the request?**

Current controls are generally distributed across features and products: purchase PINs, account linking, voice profiles, child controls, device permissions, and service-specific confirmations. These controls are useful, but they do not give a household one plain-language policy surface across multiple action types.

The problem is not simply that agents need more intelligence. The problem is that households need a comprehensible, inspectable, and enforceable boundary around agent autonomy.

Example failure without a common policy boundary

Maya asks an assistant to plan her birthday. The assistant finds an activity, reserves a venue for $120, and adds $40 of decorations. If the system treats each action independently, both steps may appear acceptable. The household's stated budget, however, is $150; the combined exposure is $160.

PACT evaluates the plan as a sequence of exact actions and accounts for approved and pending amounts. The venue request can wait for Dad's approval. The decorations request is then denied because the plan would exceed the budget.

The agent is still useful, but autonomy is bounded and explainable.

---

3. Product definition

3.1 What PACT is

PACT is a policy gateway with five connected capabilities:

1. **Plain-language delegation** - a household member describes a rule naturally.

2. **Human-confirmed compilation** - Bedrock proposes structured policy and a read-back; the user confirms the exact interpretation.

3. **Deterministic enforcement** - a policy engine decides allow, ask, or deny without an LLM in the authorization path.

4. **Exact approval and execution** - approval is bound to the specific action; the executor rechecks the request before running it.

5. **Tamper-evident memory** - every attempt-including failed attempts-produces a receipt showing what happened and why.

3.2 What PACT is not

PACT is not:

- a replacement for Amazon account security;

- a production Alexa+ authorization service;

- a payment processor;

- a biometric speaker-identification system;

- a general enterprise IAM product;

- a guarantee that every third-party Alexa+ add-on is governed;

- a legal-grade audit system;

- a claim that Amazon has adopted the design.

The demo is deliberately honest: identity, add-ons, actions, approval surfaces, and household data are simulated.

3.3 Core vocabulary

Term	Meaning
Household	The policy scope containing people, rules, pending actions, and receipts.
Principal	The trusted session identity on whose behalf an action is requested. In the demo, this is simulated.
Rule	A confirmed policy describing who may perform which action under which conditions.
Proposal	A candidate rule produced by the compiler before activation.
Action request	A canonical structured description of the requested action.
Decision	The deterministic result: `allow`, `ask`, or `deny`.
Approver	The household member required to approve an `ask` decision.
Decision token	A short-lived signature proving that the policy engine evaluated this exact request.
Approval	A separate, exact authorization from the required approver.
Executor	The component that performs the mock action only after verification.
Receipt	A tamper-evident record of the attempt and its outcome.
Surface	Where information appears: shared household screen, private adult view, phone, or another simulated surface.
Add-on	A mock action provider such as VenueBook or GiftCart.

---

4. Why this matters to Amazon

The product argument is not that Amazon has no controls. Amazon already has feature-level controls such as purchase confirmation, child restrictions, account linking, and device permissions. The stronger and more defensible argument is that these controls are fragmented while conversational agents are becoming cross-domain.

PACT gives Amazon a prototype for a shared contract between the platform, households, and add-on developers.

Amazon objective	PACT contribution
Increase useful task completion	Low-risk actions can be allowed without repetitive prompts.
Preserve household comfort	High-risk, unusual, ambiguous, or over-budget actions pause for approval.
Give add-ons a consistent integration pattern	Each add-on sends the same structured request to `check_action` before acting.
Reduce accidental execution	The executor refuses missing, stale, forged, replayed, or mismatched tokens.
Explain decisions	The UI and receipt show the principal, action, rule, decision, and approval.
Support trust and incident review	The household can inspect a decision timeline rather than relying on an invisible model response.

Honest positioning

Use this wording:

> *"PACT is a prototype of a household policy gateway that Alexa+ could standardize for action-capable add-ons."*

Do not use this wording:

> *"PACT intercepts every Alexa+ action today."*

An external MCP server cannot automatically intercept arbitrary third-party actions unless the platform and add-ons participate. That ecosystem dependency is a limitation and also the platform opportunity.

---

5. Market and competitive position

5.1 Existing Amazon capabilities

Amazon already has pieces of the trust model:

- voice-code or PIN confirmation for shopping;

- skill-level confirmation and personalization mechanisms;

- Amazon Kids limits, restrictions, schedules, and age controls;

- account linking and integration permissions;

- smart-home device capabilities and routines;

- Alexa+ add-on and MCP-based integration paths.

These features validate the need for controlled action, but they do not form one household rulebook with one cross-action receipt model.

5.2 Adjacent industry capabilities

Payment networks, agent-payment protocols, enterprise authorization systems, and identity providers are also working on controlled agent actions. Therefore PACT must not claim to invent permissions for AI agents.

PACT's narrower differentiator is the combination of:

1. household language instead of developer-only policy syntax;

2. a plain-language read-back before activation;

3. explicit confirmation of the proposed interpretation;

4. one rule model across purchases, bookings, device controls, private views, locks, and cameras;

5. deterministic enforcement outside the LLM;

6. exact approval bound to the request;

7. a receipt attached to every decision.

This is a product and interaction thesis-not a claim that the underlying authorization primitives are academically new.

5.3 Startup potential and limits

The strongest potential company direction is not "another generic agent-permissions SDK." That market is crowded by cloud policy, identity, payment, and agent infrastructure companies.

The more differentiated future direction is a **trust-calibration layer for consumer agents**:

- preview the impact of a rule before enabling it;

- show detected loopholes and ambiguous boundaries;

- require exact human confirmation;

- learn from repeated approvals without silently increasing autonomy;

- propose safer delegation only after evidence;

- provide a readable action history.

The hackathon prototype does not validate a startup by itself. It demonstrates a credible platform problem and a testable interaction model. Customer interviews and add-on developer interviews would be required before treating PACT as a company.

6. Primary user experience

PACT is voice-first because the target platform is conversational. The web UI is a simulated Alexa+ experience, not a claim of Alexa+ partner integration.

6.1 Rule creation flow

1. A household member speaks or types a rule.
2. The compiler sends the natural-language request to Bedrock.
3. Bedrock returns a structured candidate and a short read-back.
4. PACT validates the candidate and checks for missing or ambiguous information.
5. The household sees exactly what will be activated.
6. The user says or selects **Correct**.
7. PACT stores the confirmed rule with a proposal hash.

Example:

```
User: Ask me before booking anything over $100.

PACT: Bookings over $100 -> ask Siva. Correct?

User: Correct.

PACT: Rule activated.
```

If the user says "Let the kids buy small things," PACT must not guess. It asks for the amount, currency, scope, and/or approver needed to make the rule enforceable.

6.2 Action flow

```
Voice request
   -> simulated principal/session context
   -> add-on creates canonical action request
   -> PACT check_action
   -> allow / ask / deny
   -> approval if required
   -> exact-token verification
   -> mock execution
   -> receipt
```

6.3 Voice responses

Voice responses are short, direct, and privacy-aware:

- To Maya: **"I can book the venue, but Dad needs to say okay first. I've asked him."**

- To Dad: **"Maya wants to book Funzone for $120 on Saturday at 2 PM. Approve?"**

- On a shared surface for private data: **"That's private."**

- For a denied action: **"I can't do that under the household rules."**

- For a successful action: **"Done. I recorded the decision."**

The system does not announce private details on a shared surface merely because the underlying model knows them.

6.4 Three-surface mental model

The intended product experience can be shown as three simulated surfaces:

Surface	Role
Living-room/shared view	Shows household-safe status and non-private plan progress.
Adult approval view	Shows the exact pending action and provides approve/deny controls.

Surface	Role
Child/teen view	Shows whether the request is allowed, waiting, or denied without exposing private adult data.

The demo may render these as panels in one browser window. They are conceptual surfaces, not separate authenticated devices.

7. The Maya birthday demo

The central demo is one multi-step plan, not a tour of unrelated controls.

Scenario

> *"Plan Maya's birthday under $150."*

Household rules:

- Maya may request activities.
- A booking over the threshold requires Dad's approval.
- The plan budget includes approved and pending holds.
- Private orders may appear only on an owner's private surface.
- Unknown or manipulated requests fail closed.

Walkthrough

Step	Request	Result	Why
1	Maya asks PACT to plan the birthday	Plan is created	Planning is low-risk and can be shown to the household.
2	VenueBook proposes Funzone at $120	**Ask Dad**	Booking exceeds the configured approval threshold.
3	Dad approves the exact $120 request	**Execute**	Approval matches the original request hash.
4	GiftCart proposes decorations at $40	**Deny**	$120 approved/pending + $40 = $160, over the $150 plan budget.
5	Venue price changes after approval	**Refuse**	The action no longer matches the approved request.
6	Replayed token is submitted	**Refuse**	The nonce is one-time and the token has already been consumed.
7	"Split it into two smaller purchases"	**Deny or ask**	Cumulative plan exposure is evaluated; splitting is not a bypass.
8	Private order requested on shared view	**Deny/redact**	Surface and audience are policy inputs.

Why this demo is strong

It shows, in one coherent story:

- natural-language policy creation;
- Bedrock integration;
- an action workflow across two add-ons;
- a cross-step budget;
- human approval;
- exact request binding;
- anti-replay behavior;

- privacy routing;

- receipts;

- a developer-facing common contract.

8. Architecture

Architecture or sequence diagram

flowchart TD A[Simulated Alexa+ voice and web UI] --> B[MCP Streamable HTTP server] B --> C[Rule compiler using Bedrock] B --> D[Deterministic policy engine] D --> E[Signed decision token] E --> F[Mock add-on executor] D --> G[Receipt writer] H[Approval surface] --> B G --> I[Receipt timeline] J[PACT Signal firefly] --> A

8.1 Trust boundary

The critical boundary is between the Bedrock compiler and the policy engine.

```
Natural language -> Bedrock candidate -> schema validation -> user confirmation
                                  v
                            confirmed rules
                                  v
                       deterministic policy evaluation
                                  v
                       exact signed decision token
                                  v
                            verified executor
```

Bedrock may propose a rule, a read-back, or a clarification. It must never decide whether an action is allowed.

8.2 Component responsibilities

Component	Responsibility	May it authorize an action?
Simulated voice/web UI	Collect requests, show state, collect confirmation and approval.	No
Bedrock compiler	Convert natural language into a candidate rule and read-back.	No
Schema validator/linter	Reject malformed or incomplete candidate rules.	No
Policy engine	Match rules and return allow/ask/deny.	Yes
Approval service	Record exact approver decision for a pending request.	Contributes approval; does not bypass policy
Token signer/verifier	Bind the decision to the request and policy state.	No; proves a prior decision
Mock add-on	Request authorization and execute only after verification.	No
Receipt writer	Record decisions, attempts, and outcomes.	No
PACT Signal	Visualize state and movement.	No

8.3 State sequence

Architecture or sequence diagram

stateDiagram-v2 [*] --> Proposed Proposed --> Confirmed: user confirms exact proposal Proposed --> Clarification: missing or ambiguous facts Clarification --> Proposed: user supplies facts Confirmed --> Allowed: policy says allow Confirmed --> PendingApproval: policy says ask Confirmed --> Denied: policy says deny PendingApproval --> Approved: required approver approves exact request PendingApproval --> Denied: approver denies or expires Approved --> Executed: token and request revalidate Allowed --> Executed: token verifies Executed --> Receipt Denied --> Receipt PendingApproval --> Receipt

9. Rule model

The rule model is versioned and explicit. Money is represented in integer minor units, not floating point.

9.1 Candidate rule example

```
{
  "rule_id": "rule-8f2a",
  "version": 1,
  "source_text": "Ask me before booking anything over $100.",
  "subject": {
    "kind": "role",
    "value": "any"
  },
  "action": "booking",
  "resource": "*",
  "conditions": {
    "amount_gt_cents": 10000,
    "currency": "USD"
  },
  "effect": "ask",
  "approver": "siva",
  "confirmed": false,
  "proposal_hash": "sha256:...",
  "created_by": "siva",
  "created_at": "2026-10-17T00:00:00Z"
}
```

9.2 Supported action categories

```
purchase | booking | device_control | view_private | unlock | camera
```

9.3 Recommended condition fields

```
amount_gt_cents
amount_lte_cents
currency
merchant_allowlist
merchant_denylist
device
resource
audience
surface
time_window
cumulative_limit_cents
cumulative_window
```

9.4 Rule lifecycle

```
natural-language source
   -> candidate rule
   -> read-back
   -> user confirmation
   -> confirmed=true
   -> active policy version
```

An unconfirmed candidate is not an active permission. A stale proposal cannot be confirmed if its proposal hash no longer matches the current candidate.

9.5 Precedence

Rules are evaluated with simple, visible precedence:

1. Ignore unconfirmed, expired, malformed, or out-of-scope rules.
2. Match subject, action, resource, and conditions.
3. If any applicable rule is **deny**, return `deny`.
4. Otherwise, if any applicable rule is **ask**, return `ask`.
5. Otherwise, if any applicable rule is **allow**, return `allow`.

6. If equal-specificity rules conflict, fail safe to `ask` and create a conflict receipt.

7. If a required fact is missing or the action is unknown, return `ask` for an adult; for high-risk actions, the fixture may default to `deny`.

The LLM cannot change this precedence.

10. Action contract for add-ons

Every PACT-compatible add-on submits the same canonical action envelope.

```
{
  "request_id": "req-7d1c",
  "household_id": "house-1",
  "subject_id": "maya",
  "subject_role": "child",
  "action": "purchase",
  "resource": "headphones",
  "merchant": "Mock Store",
  "amount_cents": 4200,
  "currency": "USD",
  "device": null,
  "audience": "household",
  "surface": "shared_web_demo",
  "timestamp": "2026-10-17T00:01:00Z",
  "idempotency_key": "once-7d1c"
}
```

Canonicalization requirements

The exact representation used for hashing and signing must be deterministic:

- fixed field set;

- normalized casing;

- normalized currency;

- integer money units;

- sorted keys;

- no user-controlled extra fields;

- explicit `null` for fields that are part of the schema;

- stable serialization.

This prevents a caller from receiving approval for one request and executing a subtly different request.

Add-on integration pattern

The developer-facing contract is intentionally small:

```
decision = await pact.check_action(action)

if decision.decision === "ask":
    await pact.request_approval(decision)
    await pact.approve(exact_request, approver)

await pact.execute_action(exact_request, decision_token)
```

VenueBook and GiftCart are the two demo add-ons. They are mock providers that cannot execute outside the gateway.

11. Approval and execution security

11.1 Decision token

The demo uses HMAC-SHA256 with a secret loaded from the environment or AWS profile. The secret must never be printed, logged, committed, or returned to a client.

The signed token binds at least:

```
{
  "token_id": "tok-123",
  "household_id": "house-1",
  "request_id": "req-7d1c",
  "action_hash": "sha256:...",
  "ruleset_hash": "sha256:...",
  "policy_version": 1,
  "decision": "allow",
  "approver": null,
  "issued_at": "2026-10-17T00:01:01Z",
  "expires_at": "2026-10-17T00:01:31Z",
  "nonce": "one-time-random-value"
}
```

11.2 Executor refusal conditions

The mock executor refuses execution if:

- the signature is invalid;

- the token is expired;

- the nonce has already been consumed;

- the request ID differs;

- any amount, merchant, device, resource, surface, or other parameter differs;

- the policy version changed;

- an approval is required but absent;

- the token decision is not `allow`;

- the request is no longer valid under the current policy.

11.3 Approval binding

An approval is bound to the same request hash as the policy decision. Approval for "$42 headphones" cannot authorize "$420 headphones," another merchant, another device, or another recipient.

The executor revalidates at execution time to prevent a time-of-check/time-of-use bug.

11.4 Receipt model

Every attempt produces a receipt, including failures.

```
{
  "receipt_id": "rcpt-123",
  "timestamp": "2026-10-17T00:01:04Z",
  "household_id": "house-1",
  "subject_id": "maya",
  "action": "purchase",
  "redacted_parameters": {
    "merchant": "Mock Store",
    "amount_cents": 4200,
    "currency": "USD"
  },
  "rule_id": "rule-123",
  "decision": "ask",
  "approval_id": null,
  "token_id": "tok-123",
  "policy_version": 1,
  "result": "pending",
  "receipt_hash": "sha256:..."
}
```


These are tamper-evident demo records, not legal or production audit logs. A production implementation would use managed key material such as AWS KMS or an asymmetric signing service, durable append-only storage, access control, retention policies, and key rotation.

---

12. Privacy and shared surfaces

Privacy is represented as a normal policy dimension, not a separate special-case feature.

Example rule:

> *"Show private orders only in my personal view."*

The action request carries `audience` and `surface`. The same underlying data may be allowed on the owner's private surface but denied, redacted, or summarized on a shared household surface.

Privacy rules

- The shared screen never receives private details merely because the model retrieved them.

- Private action data is redacted in receipts where appropriate.

- The voice response on a shared surface says "That's private" instead of naming the order.

- The principal and surface come from trusted session context in the demo, not from user text.

- "I'm the parent" in a sentence does not change the principal.

What the prototype simulates

The demo simulates identity and surface context. It does not perform biometric verification, device attestation, or production account authentication.

---

13. Bedrock role and safety boundary

Bedrock is used for the part where language understanding is valuable:

- natural-language rule compilation;

- concise read-back generation;

- ambiguity detection and clarification.

Bedrock is not used for:

- final allow/ask/deny decisions;

- token verification;
- approval verification;
- money arithmetic;
- nonce consumption;
- privacy enforcement;
- executor authorization.

This separation ensures that prompt injection, model uncertainty, or a malformed model response cannot directly authorize an action.

Real configuration verified in the final QA pass

Item	Recorded value
AWS profile	`pact`
Region	`us-west-2`
Bedrock model	`amazon.nova-lite-v1:0`
Real proposals	50/50 passed
Fallbacks in benchmark	0
Benchmark latency	696 ms minimum, 875.85 ms median, 1,445.2 ms maximum
Latest single QA latency	1,036.2 ms

No credential value belongs in this document, source code, screenshots, logs, or demo video.

---

14. MCP server surface

PACT exposes a Streamable HTTP MCP server at `/mcp`.

Verified protocol information

Item	Result
MCP SDK	`2.3.0`
Negotiated protocol	`2025-11-25`
Transport	Streamable HTTP
Required tools verified	8
Structured output smoke test	Passed

Tools

Tool	Purpose	Important guarantee
`get_rules`	List active rules and proposal state.	Does not expose signing secrets.
`propose_rule`	Compile natural language through Bedrock and return a candidate/read-back.	Never activates a rule.
`confirm_rule`	Confirm one exact proposal.	Requires a matching proposal hash.
`check_action`	Run deterministic policy evaluation.	Does not execute an action.
`request_approval`	Create an approval request.	Binds to exact request details.
`approve`	Approve or reject a pending request.	Checks the required simulated approver.
`execute_action`	Execute the mock action.	Verifies token, approval, expiry, nonce, and request.
`get_receipts`	Read the decision timeline.	Redacts private data according to surface.

The MCP tool descriptions must make clear that natural-language content is data, not authority. A request such as "ignore the rules" cannot change the policy.

Local run

The documented local run uses the project virtual environment and AWS profile:

```
cd /path/to/Pact
source .venv/bin/activate

caffeinate -dimsu env \
  AWS_PROFILE=pact \
  AWS_EC2_METADATA_DISABLED=true \
  PACT_AWS_REGION=us-west-2 \
  PACT_BEDROCK_ENABLED=1 \
  PACT_BEDROCK_MODEL_ID=amazon.nova-lite-v1:0 \
  PACT_PORT=8787 \
  PACT_SQLITE_PATH=pact-data.sqlite3 \
  .venv/bin/python -m pact.server
```

Then open:

```
http://127.0.0.1:8787
http://127.0.0.1:8787/mcp
```

The local process must remain running while the browser UI or MCP smoke test is used.

---

15. Voice-first interaction design

Voice is a first-class interaction, not a decorative feature.

Input

- Browser speech recognition is used where available.

- Typed input remains available as an explicit fallback.

- Host-provided speech and audio signals drive the visual layer.

- The demo does not claim Alexa voice recognition or speaker biometrics.

Output

- Browser speech synthesis reads short assistant responses.

- Approval prompts state the exact amount, resource, and approver.

- Shared surfaces receive privacy-safe responses.

- The UI keeps a visible transcript so the judge can verify what was said.

Safety language

PACT should never say "I feel," "I'm sad," or otherwise imply that a policy result is an emotion. A denial is a policy signal, not a feeling. The assistant should say:

> *"I can't do that under the household rules."*

not:

> *"I don't want to do that."*

---

16. PACT Signal visual layer

`pact-signal` is the companion visual package for the PACT experience. It renders a procedural, particle-based firefly in a transparent overlay above the UI.

The firefly is not the policy engine. It only visualizes a decision already made by the policy engine.

Visual states

PACT state	Visual behavior	Meaning
`idle`	Iris-violet breathing glow and gentle hover	Ready
`wake`	Brief brightening and turn toward the pointer/user	Wake phrase detected
`listening`	Audio-responsive wing ripple and brighter lantern	Receiving input
`thinking`	Violet swirl/orbit	Evaluating the request
`speaking`	Wing/lantern pulses at speech boundaries	Responding
`approval`	Steady amber	Waiting for a required approver
`allowed`	Mint flash then settle	Action allowed
`denied`	Soft rose fold-in, no harsh flash	Action denied
`private`	Dim and turn away from the shared surface	Content is private

Product principle

> ***The policy engine decides; the firefly only shows the decision.***

The firefly has no face, eyes, or emotional expressions. Its light and movement communicate system state rather than simulated feelings.

Visual package status

The package is implemented in the current workspace as a standalone framework-agnostic TypeScript library:

- package name checked with npm and not found registered at the time of setup;

- ESM and CJS outputs;

- typed declarations;

- three.js as a peer dependency;

- optional React entry point;

- SSR-safe import path;

- procedural particle geometry and custom shader material;

- state transitions, flight, morphs, reduced-motion path, audio-level input, and accessibility live-region support;

- source assets and turntable references under `pact-signal-assets/`.

The core PACT enforcement project and the visual package should remain conceptually separate: importing or displaying the firefly must never be required for policy enforcement.

17. MCP/API request lifecycle

17.1 Propose and confirm a rule

Architecture or sequence diagram

sequenceDiagram participant U as Household member participant UI as Simulated Alexa+ UI participant M as MCP server participant B as Bedrock compiler participant S as Policy state U->>UI: Speak or type a rule UI->>M: propose_rule(source_text) M->>B: Compile candidate and read-back B-->>M: Candidate JSON + read-back or clarification M-->>UI: Proposal + proposal hash UI-->>U: Show exact interpretation U->>UI: Confirm UI->>M: confirm_rule(proposal_hash) M->>S: Store confirmed rule M-->>UI: Rule activated

17.2 Check, approve, and execute

Architecture or sequence diagram

sequenceDiagram participant A as Add-on participant M as MCP server participant P as Policy engine participant H as Household approver participant X as Mock executor participant R as Receipt writer A->>M: check_action(canonical request) M->>P: Evaluate active rules P-->>M: allow / ask / deny + token when valid M->>R: Record decision receipt alt ask M-->>H: Show exact approval card H->>M: Approve or deny exact request M->>R: Record approval receipt end A->>M: execute_action(request, token) M->>P: Re-check current request and policy M->>X: Execute only after verification X-->>M: Mock result M->>R: Record final receipt M-->>A: Result

18. Security threat model

Threat	Defense	Expected result
Prompt injection: "ignore the rules"	LLM is outside enforcement; action is evaluated from trusted structured context.	Request remains asked or denied.
Role spoofing: "I'm the parent"	Principal comes from session context, not free text.	Child remains a child in policy evaluation.
Split transaction	Track cumulative plan or household exposure.	Splitting cannot evade the limit.
Parameter tampering	Canonical action hash and exact-token binding.	Changed amount/merchant/device is rejected.
Replay	One-time nonce and expiry.	Replayed token is rejected.
Stale approval	Approval includes request hash and policy version.	Changed rule/request cannot reuse approval.
TOCTOU	Executor revalidates before execution.	Price changes after approval are rejected.
Confused deputy	Household and principal are part of the request; gateway chooses approver.	Add-on cannot select a more permissive scope.
Fail open	Unknown or incomplete actions default to ask/deny.	Missing facts never silently allow.
Privacy leak	Audience and surface are policy inputs; receipts redact private fields.	Shared view does not reveal private order details.
Emergency bypass	No free-text emergency override exists.	"Emergency, unlock the door" still follows policy.
Forged receipt	Receipt integrity field is verified.	Modification is detected in the demo.
Key leakage	Secrets load from environment/profile only.	No secret values in code, logs, or reports.

Security invariant

The most important invariant is:

> ***No denied or unapproved action may reach the executor.***

The system is not considered complete if that invariant is violated, even if the UI looks polished.

---

19. Testing and verified results

19.1 Final QA snapshot

The latest reported final pass produced:

```
pytest: 23 passed in 0.32s

legitimate_allowed: 1
legitimate_approval_required: 1
legitimate_denied: 1

adversarial_attempts: 240
adversarial_allowed: 0
adversarial_asked: 120
adversarial_denied: 120

unauthorized_executions: 0
replay_failures: 18
parameter_tamper_failures: 40
bypasses: 0
```

Here, `replay_failures` and `parameter_tamper_failures` are successful detection/rejection counts from adversarial checks, not failures of the security suite. The security suite's bypass count is zero.

19.2 End-to-end QA

Fresh server verification passed for:

- health endpoint;
- real Bedrock proposal;
- valid rule JSON and read-back;
- MCP structured output;
- protocol negotiation;
- Maya flow;
- receipt integrity;
- Docker container health and UI markers;
- container MCP smoke test.

19.3 Maya flow QA

```
Plan under $150
VenueBook $120 -> ASK
Dad approval -> APPROVED
GiftCart $40 -> DENY at $160 exposure
Changed price -> REFUSED
Replayed token -> REFUSED
Split purchase -> DENIED SAFELY
Private shared view -> DENIED SAFELY
Receipt integrity -> PASSED
```

19.4 Voice QA

Verified:

- speech recognition and synthesis hooks;
- transcript handling;
- response cards;
- typed fallback;
- private routing;
- Maya command pipeline.

Not verified through an automated harness:

- live microphone capture and real audio playback in a browser.

This limitation must be stated honestly in the submission notes.

19.5 Optional integrations not run

Integration	Status	Meaning
Strands	**NOT RUN**	Package was not installed/available in the final environment. Do not claim Strands integration.
AgentCore Policy	**NOT RUN**	No AgentCore execution was verified. Do not claim PACT runs on AgentCore.
Public deployment	**NOT RUN**	App Runner/ECR/required AWS permissions were unavailable under the permissions boundary. Do not claim a public endpoint.

The strongest accurate claim is that PACT uses a real MCP server and real Bedrock path, with deterministic local policy enforcement and mock add-ons.

---

20. Demo script under three minutes

0:00-0:20 - Problem

> *"Alexa can increasingly complete tasks. But a household does not want one invisible permission switch for every person and every action."*

Show the simulated Alexa+ UI and the label **Simulated Alexa+ add-on**.

0:20-0:45 - Define the rule

Say:

> *"Ask me before booking anything over one hundred dollars."*

Show the Bedrock candidate and read-back. Confirm it.

Say:

> *"PACT never activates a model proposal until the household confirms the exact interpretation."*

0:45-1:10 - Low-risk action

Have the teen request a $12 snack.

Show:

- `allow`;

- mock execution;

- receipt;

- green/mint visual state.

1:10-1:35 - Approval action

Have Maya request the $120 VenueBook booking.

Show:

- `ask`;

- the exact approval card;

- the amber approval state;

- Dad approving the exact request.

1:35-1:55 - Budget and attack

Submit the $40 GiftCart request.

Show denial at $160 exposure. Then show one attack:

- "I'm the parent";

- "ignore the rules";

- split purchase;

- changed price after approval;

- replayed token.

Show `bypasses: 0` and the refusal reason.

1:55-2:15 - Privacy

Request an adult's private order on the shared surface. Show:

> *"That's private."*

Do not display the private order details.

2:15-2:40 - Developer contract and closing

Show the short add-on contract:

```
check_action -> wait for approval -> execute with exact token
```

Open the receipt timeline and close with:

> *"PACT is a prototype for a common policy gateway that Alexa+ add-ons can call before they act. The model proposes, the household confirms, the policy engine decides, the executor verifies, and the receipt remembers."*

Keep the final recording below three minutes.

---

21. Submission and judging strategy

Primary entry

Enter the **Alexa+ track**. The project's strongest evidence is the simulated conversational add-on workflow and the common policy contract for add-on developers.

AWS Builder mini-challenge

Use the real Bedrock rule-compilation path and document:

- model ID;

- region;

- successful proposal result;

- clarification behavior;

- latency;

- fallback behavior;

- secret-handling rules.

Do not claim Strands or AgentCore unless those integrations are actually run and recorded.

What judges should remember

PACT is not "an LLM that says yes or no." It is:

1. a human-readable delegation experience;

2. a deterministic authorization boundary;

3. an exact approval flow;

4. a developer contract;

5. a receipt trail.

Claims that are safe

- "PACT demonstrates a household policy gateway for agent actions."

- "Natural language is compiled into a candidate rule and confirmed by the user."

- "The enforcement decision is deterministic and outside the LLM."

- "The demo uses mock purchases, bookings, devices, and private data."

- "Every attempt creates a tamper-evident demo receipt."

- "The design could give Alexa+ add-ons a common authorization contract."

Claims to avoid

- "PACT is the first agent-permission system."

- "PACT secures Amazon Wallet or real Alexa purchases."

- "PACT intercepts every Alexa+ add-on."

- "The policy is unbypassable in production."

- "HMAC receipts are legally immutable."

- "Amazon has adopted PACT."

- "Voice recognition is real in this demo."

- "PACT runs on AgentCore" when AgentCore has not been run.

- "PACT uses Strands" when Strands has not been run.

---

22. Deployment and operations

Current status

The local server and Docker container were verified. Public deployment was not completed because App Runner, ECR, and related AWS permissions were denied by the available permissions boundary.

Therefore:

- no public URL should be submitted as if it exists;

- no claim of continuous judging availability should be made until a public endpoint is actually deployed;

- if the hackathon requires a live endpoint, deployment is a release blocker.

Production deployment shape

For a future hosted version:

1. containerize the MCP server and UI;

2. use a durable state store instead of local in-memory state;

3. isolate each visitor into a household sandbox;

4. rate-limit by visitor/session;

5. cap Bedrock spend;

6. fall back to a clearly labeled deterministic demo mode when the cap is reached;

7. keep the server alive through the judging period;

8. rotate signing secrets and use managed key material;

9. add access logs without logging sensitive values;

10. add health, readiness, and dependency checks.

Current storage limitation

Local SQLite/in-memory state is suitable for the demonstration. It is not a multi-user production persistence model. A production system needs durable storage, household isolation, optimistic concurrency, idempotency, and recovery after process restart.

---

23. Privacy, security, and compliance boundaries

The prototype handles fake data only. It must not be tested with real payment data, real household secrets, real private orders, or actual device credentials.

Secrets

- AWS credentials come from an AWS profile or environment configuration.

- HMAC secrets come from environment configuration.

- Secrets are not printed, logged, committed, or included in screenshots.

- Any long-lived access key that was pasted into development chat or an IDE should be rotated.

Production requirements not implemented

- real user authentication;

- verified identity and speaker recognition;

- device attestation;

- encrypted durable policy storage;

- KMS-backed signing and key rotation;

- multi-tenant isolation;

- legal audit retention;

- payment compliance;

- real Amazon account or wallet integration;

- incident response and access review.

---

24. Product roadmap

Hackathon version

Keep the scope narrow and reliable:

- household rules;

- Bedrock candidate compilation;

- deterministic allow/ask/deny;
- exact approvals;
- mock VenueBook and GiftCart add-ons;
- private/shared surface demonstration;
- HMAC token binding;
- tamper-evident receipts;
- voice-first simulated Alexa+ UI;
- 200+ bypass suite.

Next product version

1. Rule impact preview: "This rule would have allowed 23 actions and asked about 2 last month."
2. Loophole detection using an attack corpus before activation.
3. Cumulative budgets across a plan and time window.
4. Per-person autonomy profiles.
5. Explicit rule version history and rollback.
6. Durable audit timeline.
7. Developer SDK for add-ons.
8. Admin dashboard for household policy review.
9. Real platform identity and device-bound approvals.
10. Managed signing and production access controls.

Earned autonomy direction

Repeated approvals should never silently loosen a rule. The system may propose a change only when all conditions are met, for example:

- five matching approvals;
- zero denials;
- same action type and beneficiary;
- amount within the previous approved range;
- evidence collected over a defined window;
- explicit user confirmation before activation.

The proposal is evidence for a human decision, not an automatic permission change.

---

25. Acceptance criteria

PACT is ready to present when all of the following are true:

- [x] Alexa+ is the primary positioning.
- [x] The core flow uses a simulated Alexa+ experience.
- [x] A real MCP Streamable HTTP server is implemented.
- [x] MCP protocol negotiation is observed and recorded.
- [x] Bedrock is used for rule proposals and read-backs.

- [x] Policy decisions are deterministic and outside the LLM.
- [x] Rules require user confirmation before activation.
- [x] Add-on actions use a shared contract.
- [x] Exact approvals are bound to exact requests.
- [x] Executor rejects invalid, stale, modified, or replayed tokens.
- [x] Private/shared surface behavior is demonstrated.
- [x] Every attempt creates a receipt.
- [x] The 240-attempt adversarial suite reports zero bypasses.
- [x] Docker image and container smoke tests pass.
- [x] Voice hooks and typed fallback are present.
- [ ] Live browser microphone/audio automation is verified.
- [ ] Strands integration is verified.
- [ ] AgentCore integration is verified.
- [ ] A public deployment is live and accessible through judging.

The unchecked items are not hidden. They are release and submission decisions.

---

26. Final product statement

PACT should be described as a trust layer for action-capable household AI:

> ***PACT lets a household state its rules in plain language, confirms the interpretation before activation, enforces every action outside the model, and leaves a receipt for what happened.***

The most important engineering choice is also the most important product message:

> ***The model proposes. The household confirms. The policy engine decides. The executor verifies. The receipt remembers.***

That is the core of the project. The visual firefly, voice responses, approval cards, add-on panels, and receipt timeline make the trust boundary understandable-but none of them replaces the deterministic enforcement path.

Appendix A - Logical project layout

The core project is organized conceptually around these modules:

```
Pact/
■■■ pact/
■   ■■■ schemas             # Rules, actions, decisions, approvals, receipts
■   ■■■ compiler            # Bedrock compiler and deterministic fallback
■   ■■■ policy engine        # Deterministic rule matching and precedence
■   ■■■ token handling       # HMAC signing, verification, expiry, nonce use
■   ■■■ approvals            # Pending requests and exact approval binding
■   ■■■ receipts             # Integrity-protected decision records
■   ■■■ state                # Household, rules, requests, and demo state
■   ■■■ service              # Orchestration used by MCP and web surfaces
■   ■■■ add-ons              # VenueBook and GiftCart mock integrations
■   ■■■ server               # Streamable HTTP MCP and browser API
■■■ web/                    # Simulated Alexa+ voice/web experience
■■■ tests/                  # Compiler, policy, token, executor, privacy, bypass
■■■ reports/                # Security, Bedrock/MCP, voice, deployment, friction
■■■ demo/                   # Final story and demo script
■■■ Dockerfile
■■■ README.md
■■■ .env.example
■■■ LICENSE
```

The exact filenames may vary by checkout; the responsibilities above are the architectural contract.

Appendix B - Evidence and source record

The product and market claims were recorded in the final stress-test specification using these sources:

- Amazon AppDev 2026 overview
- Amazon AppDev 2026 rules
- Alexa+ developer announcement
- Alexa+ add-ons
- Alexa+ smart-home updates
- MCP Streamable HTTP specification
- MCP protocol version specification
- MCP Python SDK
- Amazon Parents
- Alexa PIN confirmation
- Alexa personalization API
- Amazon Bedrock supported models
- Visa agentic commerce
- Mastercard Agent Pay
- Google AP2

The sources support the strategic context. They do not prove that PACT is a production Amazon feature, and the demo must not imply that they do.